Distributive Integer Linear Programming for Minimum Dominating Set Problem

Amisha Gupta, Rabea Saleh, Pappu Singh Kushwaha, and Dharmendra Prasad Mahato

Department of Computer Science and Engineering
National Institute of Technology Hamirpur
Himachal Pradesh-177 005
{24rcs011,22bcs127,24mcs112,dpm}@nith.ac.in

Abstract. In this paper, we propose a distributive integer linear programming (DILP) algorithm to solve the minimum dominating set problem. Minimum dominating set problem is NP-complete problem that arises in various domains like social networks, wireless networks and graph mining. We experiment the algorithm on real world graphs from Network Repository and compare the results with two state of the art algorithms. Results show the efficiency and reliability of the algorithm and that it can be used practically, especially for distributive large graphs. It will also be very beneficial to use this algorithm as a part of a hybrid algorithm. The code of the experiment is available at (https://www.kaggle.com/code/rabeasaleh2/ilp-for-mds).

Keywords: Minimum Dominating Set · Graph distributive algorithms · Integer Linear Programming.

1 Introduction

Given a graph $G = (V, E)$, a node $v \in V$ covers its inclusive neighborhood $N^+[v] := \{w \in V | \{v, w\} \in E\} \cup \{v\}$. a set $A \subseteq V$ covers its inclusive neighborhood $\cup_{a \in A} N^+[a]$. The set $D \subseteq V$ is a dominating set (DS) if it covers V. A dominating set of minimal cardinality is a minimum dominating set (MDS).

minimum dominating set has applications in diverse areas, especially in social networks [5], graph mining [6]and constructing a virtual backbone for the computer network[7].

minimum dominating set problem is NP-complete [8]. A greedy approximation algorithm exists, which is repeatedly choosing the node with maximum residual degree until all the nodes are covered. This algorithm achieves approximation ratio of $\ln(n) - \ln(\ln(n)) + \Theta(1)$ [9], which makes it the polynomial algorithm with the best approximation ratio for MDS problem unless $P = NP$ [10].

As an example, let us consider a graph as depicted in **Fig. 1** with eight vertices labeled as A, B, C, D, E, F, G, H.

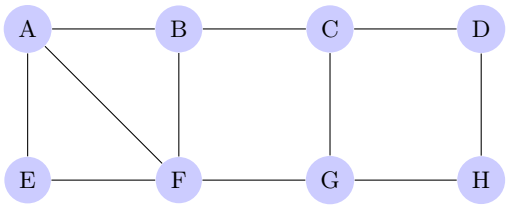

Fig. 1. Example of MDS

- Vertex F dominates itself as well as vertices A,B,E,G, .
- Vertex D dominates itself and vertices C and H.

By choosing F and D as the dominating set, all vertices in the graph are part of the set or adjacent to a vertex in it. This forms a minimum dominating set since it is a dominating set of minimal cardinality.

1.1 Our Contributions

1. We introduce a new distributive algorithm to solve the minimum dominating set problem by utilizing Integer Linear Programming in a Distributed way.
2. We Conduct experiments on 43 real-world graphs from the Network Repository, and compare the results with two state of the art algorithms.
3. We analyze the algorithm and show its benefits over the compared algorithms.

1.2 Organization of the paper

The paper is organized as follows. Section 2 descibes the related work. In Section 3, we give the intuition behind the work in this paper. Section 4 presents the proposed methodology. In section 5, we analyze the algorithm. In section 6, we describe implementation details and experimental setup. Finally, Section 7 concludes the paper.

2 Related Work

If restrictions are put on the graph, constant approximation ratio algorithms can be made and proved for different types of graphs. For example, Lenzen et al. made a distributive polynomial greedy algorithm and proved that it has a 130 approximation ratio[11], which was later proved to be 52 [12]. The best approximation ratio for a distributive polynomial Algorithm for planar graphs is 11 [13].

In practice, many heuristic algorithms are there that has a good experimental approximation ratio but doesn't have a theoritically bounded approximation ratio [2] [3] [14] [15]. Ant colony optimization [16][17][18] and

genetic algorithms [19] have also been used to solve the minimum dominating set problem. In [3], Alipour et al. made a distributive algorithm that finds a minimum total dominating set by the following way: every node i will mark a single node $j \in G[i]$ that has the maximum degree among all nodes in $G[i]$ ($G[i]$ is the adjacency list of i and $degree(i) = |G[i]|$). Next, the algorithm performs m rounds. In every round: first, every node $i \in V$ will set $degree(i)$ to be the number of times it has been marked by its neighbours in the previous round, second, every node $i \in V$ will mark a single node $j \in G[i]$ that has the maximum $degree$ among all nodes in $G[i]$. After m rounds, every node that has been marked at least once will belong to the MDS. In [2], fastDS algorithm is not distributive but is very effecient. It works by constructing initial solution by some inference rules and the well known $log|V|$ approximation ratio greedy algorithm discussed earlier.Let this initial set be D.Then,in every round, it will first remove 3 least important nodes from D and then try to add 1 or 2 nodes to D to make it a dominating set again. In every round, the size of D will reduce by one or two until it is not possible to reduce the size of D more and still make it a dominating set. More details about this can be seen in [2].

3 Intuition

Integer Linear Programming gives the exact solution to the minimum dominating set problem in non-polynomial time [1]. The time complexity of such solutions depends on the number of variables and the number of constraints in the ILP equations. If this number is small enough, the equations can be solved fast enough with ILP solvers. Here the idea of the proposed algorithm comes, what if we divide the graph into sets of connected vertices of small enough size, and solve the MDS problem independently for every set in the graph. And to insure the integration between the sets, every set i will care about covering its own members A_i but using nodes from $N^+[A_i]$ (i.e. nodes adjacent to the set A_i will also be used to cover nodes in A_i if they are part of an optimal solution (more on this will be explained in equations 1), 2), 3))).

4 Proposed Algorithm

The algorithm starts by randomly sampling $0.01|V|$ masters. This can be done distributively by the following way: every node $v \in V$ generates a random number $0 \leq p \leq 1$ and it will declare itself as a master if $p < 0.01$. Let S be the set of the sampled master nodes. Then, every node $i \in S$ will make a set $A_i = \{i\}$ and will start expanding its set and adding other nodes to its set by the following way:

- every node $i \in S$ will start by sending invite messages to $G[i]$ carrying the identifier of i and asking it to join the set A_i.
- if a node $j \in V$ that doesn't belong to any set A receives an invite message carrying the identifier of i, then:

- a) $A_i := A_i \cup j$.
- b) j will send its identifier and the identifiers of $G[j]$ to i.
- c) j will send invite message carrying the identifier of i to every $k \in G[j]$.

– if a node $j \in V$ that does belong to some set A receives an invite message, it will simply drop it.

Assuming that the graph is connected, after network finishes exchanging messages, every node $v \in V$ will belongs to exactly one set A. Further, every node $i \in S$ will have complete information about $G[j] \forall\, j \in A_i$. The average size of A_i is approximately 100. Next, the computer of every node $i \in S$ will solve the following integer linear programming equation:

$$minimize \sum_{j \in N^+[A_i]} x_j$$

$$constrains: \; x_j + \sum_{k \in G[j]} x_k \geq 1 \quad \forall\, j \in A_i$$

$$x_j \in \{0, 1\} \quad \forall j \in N^+[A_i]$$

$x = 1$ means that $x \in MDS$, and $x = 0$ otherwise

Algorithm 1 DILP Algorithm

Input: Graph $G(V, E)$
Output: minimum dominating set MDS
Initialization: set S and sets $A_i = \{i\}$ for every $i \in S$
for every node $i \in S$ in parallel **do**
  1-expand the set A_i to include as much vertices as possible
  2-solve the ILP equations (1,(2,(3 and inform the nodes that will belong to MDS
end for
return MDS

5 Analysis

This algorithm can run distributively (unlike fastDS [2]) which can be very beneficial for extremely large graphs. Also, this algorithm requires synchronization only once when forming the groups (sets) in step 1 of the Algorithm as described previously. nodes need to wait for some time until all invite messages have been dropped. Note that on average, every invite message i will reach to 100 nodes before it is dropped everywhere in the network. the time taken by this step will not be dependent on the total size of the graph.

Then, every set can solve its problem in asynchronous way. Alipour et al. [3] algorithm performs several rounds and synchronization is strictly required in all the rounds for the algorithm to terminate correctly. This is because the degree of every vertix needs to be updated after every round, so nodes have to wait until this updation finishes for all nodes in the graph before starting a new round, otherwise the results will not be correct. This makes the proposed algorithm also better for extremely large graphs where synchronization can be a problem.

The sizes of the sets $A_i, i \in S$ will be 100 on average, but it can reach to any arbitrarily large size. To mitigate this problem, all the nodes in the graph should have previous knowledge about the time at which they have to start the algorithm, so that the propagation of the invite messages from the masters will happen as parallel as possible to ensure that $|A_i|$ is nearest possible to 100 for every $i \in S$. Whenever a node starts the algorithm (by generating the random number p) , if it is a master, it has to set a time limit T. after T finishes, it can assume that all its invite messages have been dropped and all messages carrying information from its set members have reached to it (to the master) and now it can start forming and solving the ILP equations asynchronously from other masters.

The independence between the sizes of the sets $A_i, i \in S$ and the total size of the graph will results into the independence between the total time required by the algorithm to finish and the size of the graph.

6 Implementation and Experimentation

We simulated the algorithm in python on kaggle (CPU, 30 Giga bytes RAM), code is available here (https://www.kaggle.com/code/rabeasaleh2/ilp-for-mds). the cell of the code of the algorithm has the comment "main code for DILP" above it.
Breadth First Search is used to simulate the parallel propagation of invite messages. $PULP_CBC_CMD$ is used to solve the ILP equations. Let $time(i)$ denotes the time taken by the master i to solve the ILP equations of its group, then the running time of the algorithm for one experiment is measured as

$$\max_{i \in S}(time(i))$$

and the time in Table 1 is averaged over all experiments (the number of experiments is specified by *repeated* value). We compare our algorithm with two state of the art algorithms, Alipour et al. [3] and fastDS [2] on 43 real world graphs from Network Repository [4]. It is to be noted that Alipour et al.

[3] algorithm actually calculates a minimum total dominating set (which is indeed a minimum dominating set). The results are shown in table 1. All times are in seconds. The time in Alipour et al. [3] is the running time for the centralized version of the algorithm. *repeated* stands for the number of times the experiment was repeated on this graph (random seeds are 1,2,3,...). avg $|A_i|$ stands for average group size over all experiments on this graph in DILP. max $|A_i|$ stands for maximum group size over all experiments on this graph in DILP. *beated* is equal to 3 if DILP algorithm beats Alipour et al. [3] and 2 if it beats fastDS [2] in the size of the MDS found.

DILP algorithm assumes that the graph is connected, which is not the case in

Table 1. Experimental results on Network Repository

instance	time(s)m=0,2,5[3]	time(s)DILP	$\|MDS\|$[3]	$\|MDS\|$fastDS[2]	$\|MDS\|$DILP	repeated	avg$\|A_i\|$	max$\|A_i\|$	beated
amazon0302(V262K E1.2M)	4.322, 11.832, 21.681	8.529	82690,71023,68914	35593	52047.6	10	100.567	10759	3
socfb-A-anon(V3M E24M)	79.337, 185.572, 333.808	51.521	231903,211315,203021	201690	254728.7	10	100.336	82562	-
p2p-Gnutella31(V62K E147K)	0.174, 0.496, 1.057	0.325	13917, 13123, 13086	12582	13706	10	101.868	2859	-
p2p-Gnutella30(V36K E88K)	0.065, 0.201, 0.308	0.313	8067, 7559, 7379	7169	7928.2	10	103.328	1851	-
p2p-Gnutella25(V22K E54K)	0.033, 0.084, 0.150	0.218	5066, 4771, 4637	4519	5074.6	10	101.91	2151	-
co-papers-dblp(V540K E15M)	3.601 , 9.422, 9.422	3.898	62670 , 52802 , 52196	43978	45876.4	10	100.609	5034	3
soc-sign-Slashdot0902211(V82K E549K)	0.381, 0.817, 1.383	8.735	16924, 16331, 16117	-	16573.2	10	101.754	26293	-
p2p-Gnutella24(V26K E65K)	0.039, 0.117, 0.208	0.515	6056, 5724, 5569	5418	6068.6	10	102.100	4486	-
soc-Epinions1(V75K E508K)	2.050, 2.589, 3.298	8.160	17146, 16646, 16132	15734	16377.8	10	101.960	32773	-
amazon0601(V403K E3.3M)	6.844,15.708, 26.702	29.063	85559, 70372, 66267	42289	66228.6	10	100.544	35185	3
rgg-n-2-19-s0(V524K E3M)	14.948, 37.951, 69.568	12.472	127417, 88424, 86640	44423	55534.7	10	100.436	781	3
amazon0312(V400K E3.2M)	7.833,19.076, 32.826	20.556	100960, 81107, 75368	45490	68980.8	10	100.587	22578	3
co-papers-citeseer(V434K E16M)	2.253 , 6.214 , 11.798	3.356	43879 , 37352 , 37043	26082	31646.2	10	100.417	6105	3
soc-sign-Slashdot090216(V81K E545K)	0.198, 0.487, 0.845	4.811	17291, 16290, 15871	15305	16537.7	10	101.736	25497	-
coAuthorsCiteseer(V227K E814K)	1.322, 3.682, 7.144	0.641	40878, 38444, 38331	22011	36178.3	10	100.383	7339	3
web-NotreDame(V325K E1.5M)	2.301, 6.293, 11.914	40.479	34788, 31929, 30883	23735	25167.5	10	100.5	60171	3
amazon0505(V410K E3.3M)	6.962,16.653, 29.229	582.286	91009, 78355, 72273	47310	70903.9	10	100.4	25244	3
soc-sign-Slashdot081106(V77K E516K)	0.198, 0.487, 0.845	5.306	16197, 15184, 15069	14312	15346.5	10	101.9	25677	-
rgg-n-2-22-s0 (V4.1M E30.3M)	862.451, 2127.245, 3918.068	11.593	952637, 635144, 619729	312350	398463.5	10	100.1	610	3
rgg-n-2-17-s0 (V131K E728K)	1.101, 2.649, 4.817	2.102	33789, 24020, 23595	43412	15072.0	10	101.9	517	2,3
rgg-n-2-20-s0(V1M E6.8M)	57.436,143.619, 265.936	5.529	248861, 170092, 166453	84708	106839.4	10	100.3	549	3
rgg-n-2-21-s0(V2M E14.4M)	222.205, 554.531, 1021.933	9.450	487626, 328695, 320941	162266	206191.1	10	100.1	713	3
ca-coauthors-dblp(V540K E15M)	3.679, 9.324, 17.806	2.956	62656, 52826, 52237	35597	45876.4	10	100.6	5034	3
sc-pwtk (V218K E6M)	0.714, 2.236, 4.396	0.269	9088, 8552, 8482	4200	7070.3	10	100.4	570	3
soc-pokec (V2M E22M)	46.195, 117.618, 215.056	71.531	301500, 241907, 239900	207308	326569.0	1	101.3	24819	-
ca-hollywood-2009(V1.1 E56.3)	11.049, 26.983, 46.099	241.019	75430, 62699, 61658	48740	77402.4	5	100.7	22478	-
ca-dblp-2012(V317K E1M)	2.649, 7.648, 14.942	12.079	55071, 51946, 51841	46138	51484.8	5	100.6	11779	3
rt-retweet-crawl(V1M E2M)	12.787, 37.394, 72.781	12.995	84722, 82386, 81350	75740	77485.2	5	100.7	72691	3
inf-roadNet-CA(V2M E3M)	405.566, 1219.695, 2436.119	1465.424	900458, 856218, 841233	586513	619357.4	5	100.4	798	3
soc-digg(V771K E6M)	7.195, 19.044, 35.899	577.569	76454, 71018, 68846	66155	74499.2	5	101.0	97181	-
soc-buzznet(V101K E3M)	0.216, 0.543, 0.932	63.2	201, 145, 138	127	755.8	5	102.1	86140	-
sc-shipsec1(V140K E2M)	0.748, 1.871, 3.359	243.2	17577, 13273, 11863	7662	12528.6	3	100.3	467	-
inf-roadNet-PA(V1M E2M)	125.252, 377.544, 750.686	2326.3	528121, 501972, 498389	326934	345380.6	3	100.7	745	3
soc-youtube(V496k E2M)	4.103, 12.808, 24.979	595.7	107611, 102464, 102366	89732	97071.6	3	101.3	235172	3
soc-flickr(V514K E3M)	7.190, 22.538, 48.541	110.2	111049, 106455, 106326	98062	103621.0	3	101.3	84151	3
socfb-B-anon(V3M E21M)	70.376, 165.034, 301.235	55.6	215288, 190114, 188315	187030	227746.0	3	100.7	41928	-
soc-flixster(V3M E8M)	25.354, 73.153, 142.177	23.231	92197, 91516, 91010	91019	100749.3	3	100.5	71965	-
soc-youtube-snap(V1M E3M)	27.546, 80.038, 153.958	1269.6	235870, 228008, 224933	213122	221837.0	3	100.6	289877	3
soc-FourSquare(V639K E3M)	18.054, 28.809, 44,000	3468.2	63643, 62245, 62050	60979	64826.6	3	101.0	402363	-
soc-lastfm(V1M E5M)	8.326, 24.161, 45.156	19.51	68060, 67696, 67138	67226	71408.0	3	100.47	41049	-
soc-livejournal(V4M E28M)	459.524, 1230.058, 2317.355	68.405	903243, 861908, 853903	793887	954393.4	5	100.3	59494	-
soc-delicious(V536K E1M)	3.708, 11.756, 22.703	10.15	58614, 57596, 55840	55722	59462.0	5	101.1	59337	-
sc-rel9(V6M E24M)	257.363, 731.034, 1375.043	9.847	230045, 201699, 197014	127548	225269.2	5	100.3	11532	-

all network repository instances. We removed the instances from the table where the resulted dominating set does not cover all V with a tolerance of total of < 100 uncovered nodes (we removed 7 instances from 50 and remained with 43 instances). This happens because the size of some coneected components was small enouph (< 400) that no master will be announced in those connected components. Such a problem can be mitigated by choosing larger values of p (like 0.1 instead of 0.01).
Overall, the results of the experiments have very low standard deviation. The results of every experiment alone and more informations can be found in this file (https://drive.google.com/file/d/1Ks1MYPkqyOZtLOYk-udYPKnS-mjw8kKG/view?usp=sharing).
Out of 43 instances, DILP performs better than Alipour et al.[3] algorithm in 22 instances and better than fastDS[2] in 1 instance. DILP has more chance to perform better than Alipour et al.[3] when the graph is large compared to small graphs.

7 Conclusion

In this paper, we made a distributed integer linear programming algorithm (DILP) that solves the minimum dominating set problem and showed its performance practically in comparison to two state-of-the-art algorithms Alipour et al.[3] and fastDS[2] on 43 instances of Network Repository[4] graphs. Results showed the practical benefit of this algorithm as it can be implemented distributively and it gives smaller minimum dominating set size compared to Alipour et al.[3] algorithm in less time in many instances. This suggests that it can be used as a part of a hybrid algorithm that combines more than one algorithm and takes the solution with the smallest size or the first solution that comes (the fastest one).

8 References

[1] Johan M. M. van Rooij and Hans L. Bodlaender. 2011. Exact algorithms for dominating set. Discret. Appl. Math. 159, 17 (2011), 2147–2164. https://doi.org/10.1016/j.dam.2011.07.001

[2] Shaowei Cai, Wenying Hou, Yiyuan Wang, Chuan Luo, and Qingwei Lin. 2020.Two-goal Local Search and Inference Rules for Minimum Dominating Set. In Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI 2020. 1467–1473. https://doi.org/10.24963/ijcai.2020/204

[3] Alipour, S., Futuhi, E. and Karimi, S. On distributed algorithms for minimum dominating set problem, from theory to application. arXiv preprint arXiv:2012.04883 (2020).

[4] Ryan A. Rossi and Nesreen K. Ahmed. The network data repository with interactive graph analytics and visualization. In Proceedings of AAAI 2015, pages 4292–4293,2015.

[5]FengWang,ErikaCamacho, andKuaiXu. Positive influence dominating set in online social networks. In Proceedings of COCOA 2009, pages 313–321, 2009.

[6]David Chalupa. An order-based algorithm for minimum dominating set with application in graph mining. Inf. Sci., 426:101–116, 2018.

[7]Jasaswi Prasad Mohanty, Chittaranjan Mandal, Chris Reade, Ariyam Das, Construction of minimum connected dominating set in wireless sensor networks using pseudo dominating set, Ad Hoc Networks, Volume 42, 2016, Pages 61-73, ISSN 1570-8705, https://doi.org/10.1016/j.adhoc.2016.02.003.

[8] Richard M. Karp. 1972. Reducibility Among Combinatorial Problems. In Proceedings of a symposium on the Complexity of Computer Computations, held March 20-22, 1972, at the IBM Thomas J. Watson Research Center, Yorktown Heights, New York, USA (The IBM Research Symposia Series), Raymond E.Miller and James W. Thatcher (Eds.). Plenum Press, New York, 85–103. https://doi.org/10.1007/978-1-4684-2001-2_9

[9]Petr Slavík. 1996. A tight analysis of the greedy algorithm for set cover. In Proceedings of the twenty-eighth annual ACM symposium on Theory of Computing (STOC '96). Association for Computing Machinery, New York, NY, USA, 435–441. https://doi.org/10.1145/237814.237991

[10] Irit Dinur and David Steurer. 2014. Analytical approach to parallel repetition. In Proceedings of the forty-sixth annual ACM symposium on Theory of computing (STOC '14). Association for Computing Machinery, New York, NY, USA, 624–633. https://doi.org/10.1145/2591796.2591884

[11] Lenzen, C., Pignolet, Y.-A., Wattenhofer, R.: Distributed minimum dominating set approximations in restricted families of graphs. Distrib. Comput. 26(2), 119–137 (2013)

[12] Wawrzyniak, W.: A strengthened analysis of a local algorithm for the minimum dominating set problem in planar graphs. Inf. Process.Lett. 114(3), 94–98 (2014)

[13] Ozan Heydt, Simeon Kublenz, Patrice Ossona de Mendez, Sebastian Siebertz, Alexandre Vigny,Distributed domination on sparse graph classes,European Journal of Combinatorics,Volume 123,2025,103773,ISSN 0195-6698,

https://doi.org/10.1016/j.ejc.2023.103773

[14] Laura A. Sanchis. 2002. Experimental Analysis of Heuristic Algorithms for the Dominating Set Problem. Algorithmica 33, 1 (2002), 3–18. https://doi.org/10.1007/s00453-001-0101-z

[15] A. Casado, S. Bermudo, A.D. LÃ³pez-SÃ¡nchez, J. SÃ¡nchez-Oro,An iterated greedy algorithm for finding the minimum dominating set in graphs,Mathematics and Computers in Simulation,Volume 207,2023,Pages 41-58,ISSN 0378-4754,
https://doi.org/10.1016/j.matcom.2022.12.018.

[16] Anupama Potluri, Alok Singh, Two Hybrid Meta-heuristic Approaches for Minimum Dominating Set Problem, 2011, pp. 97–104.

[17]Anupama Potluri, Alok Singh, Hybrid metaheuristic algorithms for minimum weight dominating set, Appl. Soft Comput. 13 (2013) 76–88.

[18]David Chalupa, An order-based algorithm for minimum dominating set with application in graph mining, Inf. Sci. 426 (2018) 101–116.

[19]Abdel-Rahman Hedar and Rashad Ismail. 2010. Hybrid Genetic Algorithm for Minimum Dominating Set Problem. In Computational Science and Its Applications - ICCSA 2010, International Conference, Fukuoka, Japan, March 23-26, 2010,Proceedings, Part IV. 457–467. https://doi.org/10.1007/978-3-642-12189-0_40